# Quality control — Study B (contraception)

**What this document is.** Before we test anything, we look hard at the raw data and ask: are the three files consistent with each other, are there samples that behave strangely, and is anything broken badly enough that we should drop or fix it now? Every number below was computed from the data — at the bottom of each section there is a *sanity check* telling you how to spot-check it yourself.

```
library(tidyverse)
library(patchwork)   # puts several ggplots in one row

# Workshop helper: read_counts() loads the three count tables plus metadata
# and cytokines, and stops with an error if the sample ids do not line up.
source(here::here("R/count_tables.R"))

d  <- read_counts("group_b")      # the contraception study
sp <- d$species                   # 220 species  x 93 samples
ge <- d$genes                     # 14,226 genes x 93 samples
md <- d$metadata                  # one row per sample
cy <- d$cytokines                 # 15 immune markers, one row per sample
```

## 1. Do the pieces fit together?

The most dangerous QC failure is silent mis-alignment: if the count-table columns and the metadata rows were in a different order, every test would be wrong and nothing would warn us. `read_counts()` refuses to return if the sample ids of the two count tables and the metadata are not exactly the same set, and it reorders the columns to match the metadata — but it does **not** check the cytokine table, so we check all four tables ourselves here, both for the same *set* of ids and for the same *order*.

```
# Every cell is turned into text, so one column can hold counts and yes/no answers.
tibble(
  check = c(
    "samples in metadata",
    "species table: features x samples",
    "gene table: features x samples",
    "cytokines rows",
    "sample_id appears more than once?",
    "participant appears more than once?",
    "species ids same set as metadata?",
```

```
    "gene ids same set as metadata?",
    "cytokine ids same set as metadata?",
    "species columns ordered like metadata?",
    "gene columns ordered like metadata?",
    "cytokine rows ordered like metadata?",
    "negative or fractional counts anywhere?",
    "non-integer counts?"
  ),
  result = c(
    nrow(md),
    str_c(nrow(sp), " x ", ncol(sp)),
    str_c(nrow(ge), " x ", ncol(ge)),
    nrow(cy),
    anyDuplicated(md$sample_id),
    anyDuplicated(md$participant_id),
    setequal(md$sample_id, colnames(sp)),
    setequal(md$sample_id, colnames(ge)),
    setequal(md$sample_id, cy$sample_id),
    identical(colnames(sp), md$sample_id),
    identical(colnames(ge), md$sample_id),
    identical(cy$sample_id, md$sample_id),
    any(sp < 0) || any(ge < 0),
    any(sp != round(sp)) || any(ge != round(ge))
  ) |>
    as.character()
)
```

```
# A tibble: 14 × 2
   check                                result
   <chr>                                <chr>
 1 samples in metadata                  93
 2 species table: features x samples    220 x 93
 3 gene table: features x samples       14226 x 93
 4 cytokines rows                       93
 5 sample_id appears more than once?    0
 6 participant appears more than once?  0
 7 species ids same set as metadata?    TRUE
 8 gene ids same set as metadata?       TRUE
 9 cytokine ids same set as metadata?   TRUE
10 species columns ordered like metadata? TRUE
11 gene columns ordered like metadata?  TRUE
12 cytokine rows ordered like metadata? TRUE
```

```
13 negative or fractional counts anywhere? FALSE
14 non-integer counts?                      FALSE
```

**Sanity check.** The workshop materials say Study B has 93 samples. Both tables should say `x 93`. One participant should give one sample — the duplicates check must be `0`, and all six id checks should say `TRUE`. To verify one of them yourself: open `06_cytokines.csv`, sort its first column, and confirm the first and last entries are `B001` and `B093`, just as in `01_metadata.csv`.

## 2. Who is in the study, and is it balanced?

Downstream tests assume the groups we compare are big enough and not wildly lopsided. Also worth checking now: is `rnaseq_batch` (which sequencing run a sample was processed in) mixed evenly across the arms, or confounded with them? If batch sat entirely inside one arm we could never separate "the pill changed the genes" from "the second run did".

```
knitr::kable(table(Arm = md$arm, `RNA-seq batch` = md$rnaseq_batch))
```

| | B1 | B2 |
|---|---|---|
| injectable | 20 | 19 |
| pill | 14 | 14 |
| ring | 13 | 13 |

All 93 participants contribute exactly one sample. The three arms (93 samples: 28 pill, 39 injectable, 26 ring) are reasonably close in size and both sequencing batches contain samples from all three arms, so batch can be included as a covariate without swallowing the treatment.

Study B has a single visit per participant, so there is no time point to cross-tabulate against `rnaseq_batch` — that check belongs to Study A. What we can ask here is the Study B version of "did the arms differ at baseline?": with one visit, every metadata column is a baseline measurement, so we compare them across arms.

```
knitr::kable(table(Arm = md$arm, `BV status` = md$bv))
```

| | intermediate | negative | positive |
|---|---|---|---|
| injectable | 0 | 20 | 19 |
| pill | 2 | 16 | 10 |
| ring | 1 | 15 | 10 |

```
knitr::kable(table(Arm = md$arm, `Any STI` = md$any_sti))
```

| | 0 | 1 |
|---|---|---|
| injectable | 20 | 19 |
| pill | 16 | 12 |
| ring | 9 | 17 |

```
tibble(
  variable = c("Nugent score", "pH"),
  test     = c("Kruskal–Wallis", "Kruskal–Wallis"),
  p_value  = signif(c(kruskal.test(nugent ~ arm, md)$p.value,
                      kruskal.test(ph ~ arm, md)$p.value), 2)
) |> knitr::kable()
```

| variable | test | p_value |
|---|---|---|
| Nugent score | Kruskal–Wallis | 0.63 |
| pH | Kruskal–Wallis | 0.02 |

BV status (Fisher's exact p = 0.45) and STI history (p = 0.22) are distributed similarly across arms, and Nugent scores are too (median 2 in pill and injectable, 2.5 in ring). **Vaginal pH does differ** (Kruskal–Wallis p = 0.02; median 4.55 in the pill arm vs 5.0 in injectable and ring, and the ring arm has no sample below 4.4). The shift is small and could easily be a chance imbalance among 26–39 people, but because pH tracks BV and shifts with contraceptive hormones in the literature, we will keep it in mind — and consider adjusting for it — when the arms are compared on the microbiome.

## 3. Missing values

Count tables almost never have missing values (an unmeasured gene just gets zero reads), so missingness lives in the spreadsheet-like tables. For the metadata the answer is good news: **not a single cell is missing** — every one of the 8 columns (`sample_id`, `participant_id`, `arm`, `bv`, `nugent`, `ph`, `any_sti`, `rnaseq_batch`) is filled for all 93 samples, so no sample has to be dropped for lack of arm or batch information and no variable needs imputation or a "missing" category:

```
bind_rows(
  metadata  = tibble(source = "metadata",  missing = sum(is.na(md)),  cells =
nrow(md) * ncol(md)),
  cytokines = tibble(source = "cytokines", missing = sum(is.na(cy)), cells =
nrow(cy) * ncol(cy))
) |>
  mutate(percent = round(100 * missing / cells, 2))
```

```
# A tibble: 2 × 4
  source    missing cells percent
  <chr>       <int> <int>   <dbl>
1 metadata        0   744    0
2 cytokines      30  1488    2.02
```

So the only missing data in the study is here: 30 of 1,395 cytokine cells. Which samples?

```
md_missing <- md$sample_id[rowSums(is.na(cy[, -1])) > 0]
md_missing
```

```
[1] "B018" "B029"
```

**Finding: `B018` and `B029` are missing all 15 cytokines** (the columns are skipped, not partly filled), which reads like an assay that failed for two samples. Nothing to impute — they simply have no immune measurements. We keep them for the microbiome and gene analyses and lose them only in cytokine work.

> **Sanity check.** `is.na(cy)` should count exactly 30 (2 samples × 15 markers), and the two ids above should appear in `01_metadata.csv` with all other fields filled in.

## 4. Library sizes: how much did we sequence?

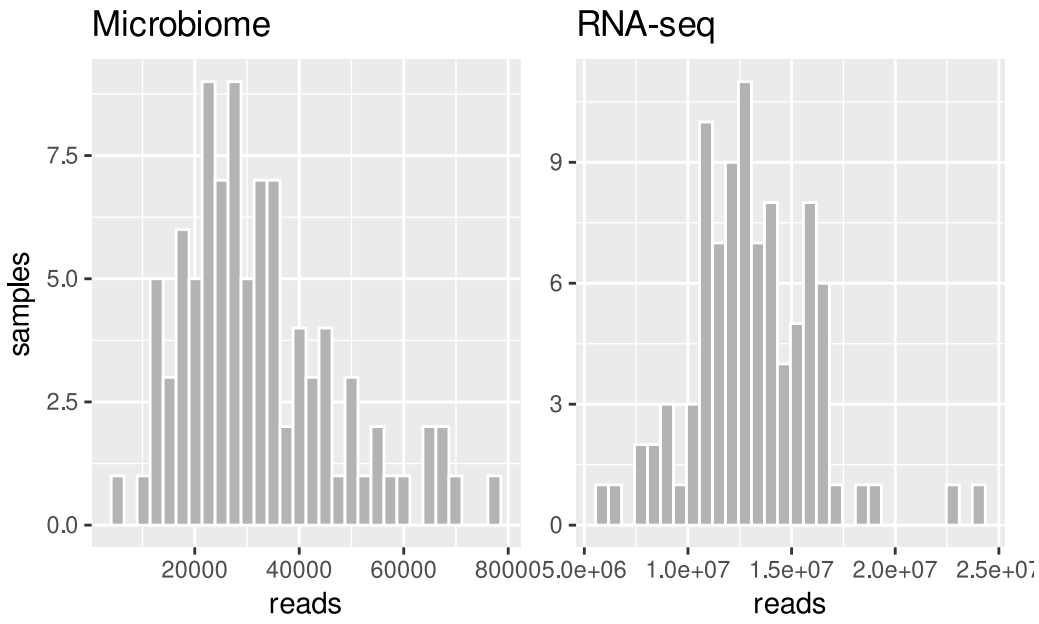
"Library size" means the total number of reads mapped for a sample — literally the column sum of the count table. Samples far below the rest may just have been sequenced shallowly, which makes them noisy, not broken.

```
qc <- bind_cols(
  md,
  lib_species = colSums(sp),
  lib_genes   = colSums(ge)
)

p_sp <- ggplot(qc, aes(lib_species)) +
  geom_histogram(bins = 30, fill = "grey70", colour = "white") +
  labs(title = "Microbiome", x = "reads", y = "samples")

p_ge <- ggplot(qc, aes(lib_genes)) +
  geom_histogram(bins = 30, fill = "grey70", colour = "white") +
  labs(title = "RNA-seq", x = "reads", y = NULL)

p_sp + p_ge
```




```
# The rules below are our choices, not universal constants — see "Choices made".
rule <- median(qc$lib_species) / 2
qc |> filter(lib_species < rule) |> select(sample_id, lib_species) |>
  knitr::kable(digits = 0, caption = str_c("Microbiome: samples below half the
median library size (", format(rule, big.mark = ","), " reads)"))
```

| sample_id | lib_species |
|---|---|
| B003 | 14057 |
| B006 | 5247 |
| B024 | 11627 |
| B065 | 11570 |
| B069 | 12927 |
| B070 | 13333 |
| B076 | 14534 |
| B083 | 10756 |
| B091 | 13543 |

Species libraries run from about 5,000 to 77,000 reads (median 29,000), and nine samples fall below half the median — `B006` is the lowest at 5,247 reads, roughly six times fewer reads than the median sample. None is so low that it must be dropped; we flag them and will look again after ordination

in case any sits alone. Gene libraries are around 6–24 million reads, tightly packed — only one sample sits below half the median there, **B042 at 5.9 million** reads, and only barely (the rule is 6.4 million). B042 is not one of the nine low-depth microbiome samples, so its shallow sequencing is specific to the RNA-seq library, not to the sample as a whole.

> **Sanity check.** Add the first five columns of any sample of your choice in `02_microbiome_counts.csv` with a calculator — it should match `qc$lib_species[qc$sample_id == "B001"]` (or whichever sample you picked).

## 5. Dominance: is one feature eating each sample?

Both tables are **compositional**: sequencing gives us proportions, not absolute amounts, so every table must sum to the library size. That means a huge read count for one feature mechanically shrinks all the others. Some dominance is expected — vaginal communities are often 90 % one *Lactobacillus* — but near-total domination is worth seeing explicitly.

```
top_share <- function(m) apply(m, 2, max) / colSums(m)
qc$top_species <- top_share(sp)
qc$top_gene    <- top_share(ge)

worst_sp <- qc |> slice_max(top_species, n = 3) |>
  transmute(sample_id, share = str_c(round(100 * top_species, 1), "%"))

knitr::kable(worst_sp, caption = "Samples whose single most abundant species
takes the largest share")
```

| sample_id | share |
|---|---|
| B030 | 99.8% |
| B016 | 99.7% |
| B012 | 99.7% |

The median sample's biggest species is about half of all reads — normal for this body site. The extreme is B030, which is 99.8 % *Lactobacillus iners*. That is not an error: some women really are monoculture, and it is exactly the biology we want to study. We keep it, but we will not be surprised later if it pulls ordinations towards itself.

> **Sanity check.** In `02_microbiome_counts.csv`, find the B030 column, sort it, and confirm the biggest number is about 99.8 % of the column sum.

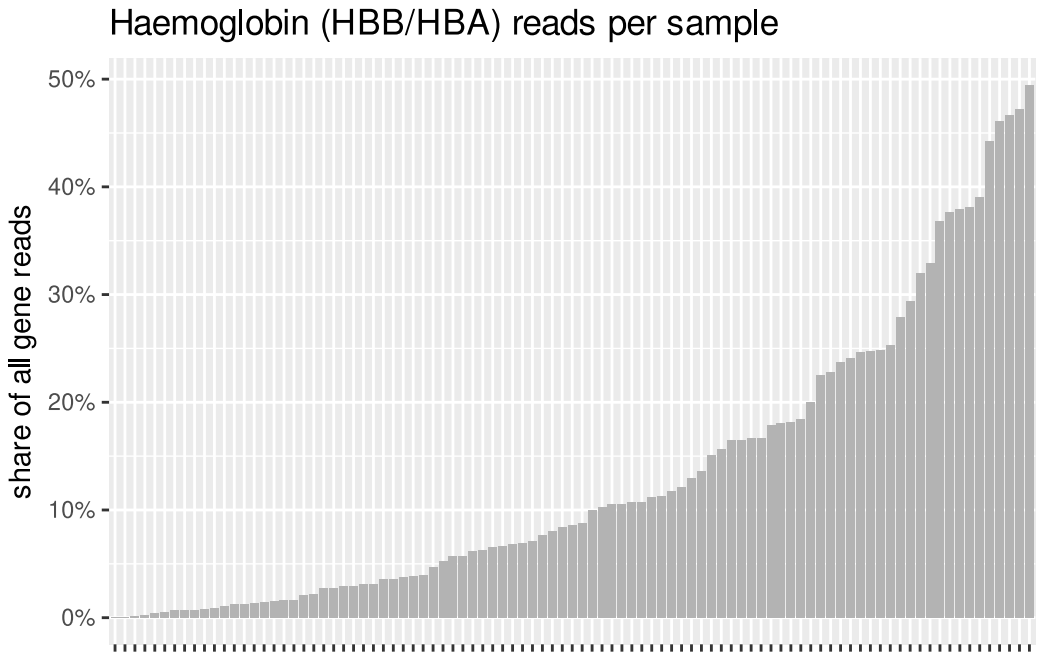
## 6. Anything odd inside the RNA-seq?

The largest gene share in any sample is 29% — in B093, nearly a third of all reads come from **HBB**, the beta chain of haemoglobin. Haemoglobin genes are the classic signature of blood contami-

nation in mucosal swabs: the sample picked up red blood cells, and RNA from blood crowded out cervical/vaginal transcripts. It is worth remembering when interpreting `B093` specifically, but a quarter of samples carry some HBB signal and the difference is quantitative, not broken, so we keep the sample.

```
# Proportion of each sample's reads belonging to the haemoglobin genes HBB/HBA
hb <- d$gene_info |> filter(symbol %in% c("HBB", "HBA1", "HBA2")) |>
pull(ensembl_gene_id)
qc$hbb_share <- colSums(ge[hb, , drop = FALSE]) / qc$lib_genes

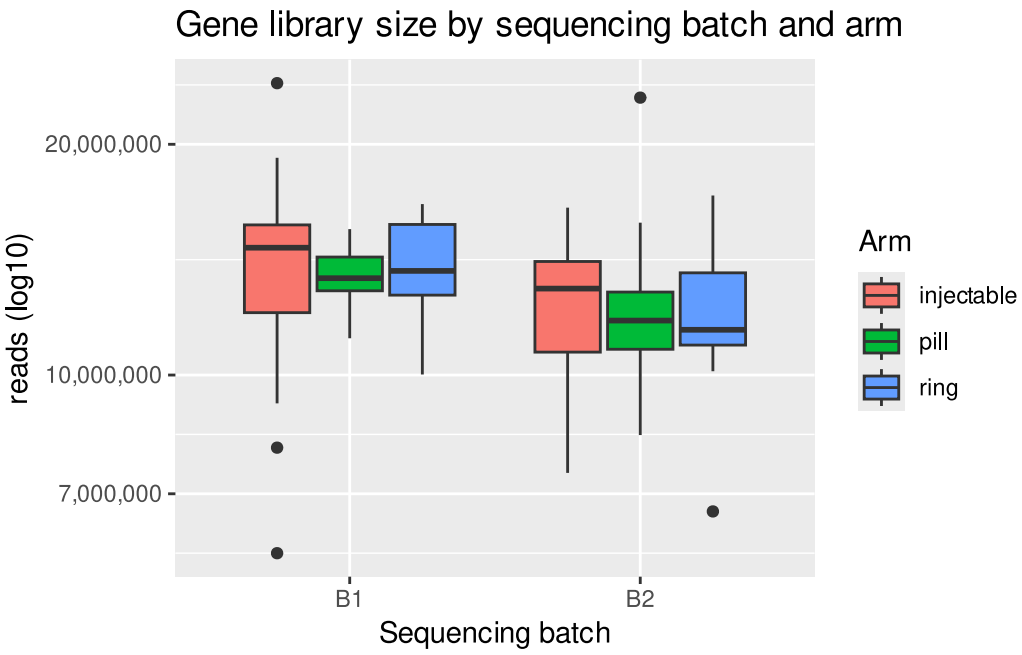
ggplot(qc, aes(reorder(sample_id, hbb_share), hbb_share)) +
  geom_col(fill = "grey70") +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Haemoglobin (HBB/HBA) reads per sample",
       x = NULL, y = "share of all gene reads") +
  theme(axis.text.x = element_blank())
```




**Sanity check.** Sort one sample's column in `04_rnaseq_counts.csv.gz` (unzip it first) by value; the top gene for `B093` should be HBB.

## 7. Batch effects at a glance

```
ggplot(qc, aes(rnaseq_batch, lib_genes, fill = arm)) +
  geom_boxplot() +
  scale_y_log10(labels = scales::comma) +
  labs(title = "Gene library size by sequencing batch and arm",
       y = "reads (log10)", fill = "Arm", x = "Sequencing batch")
```

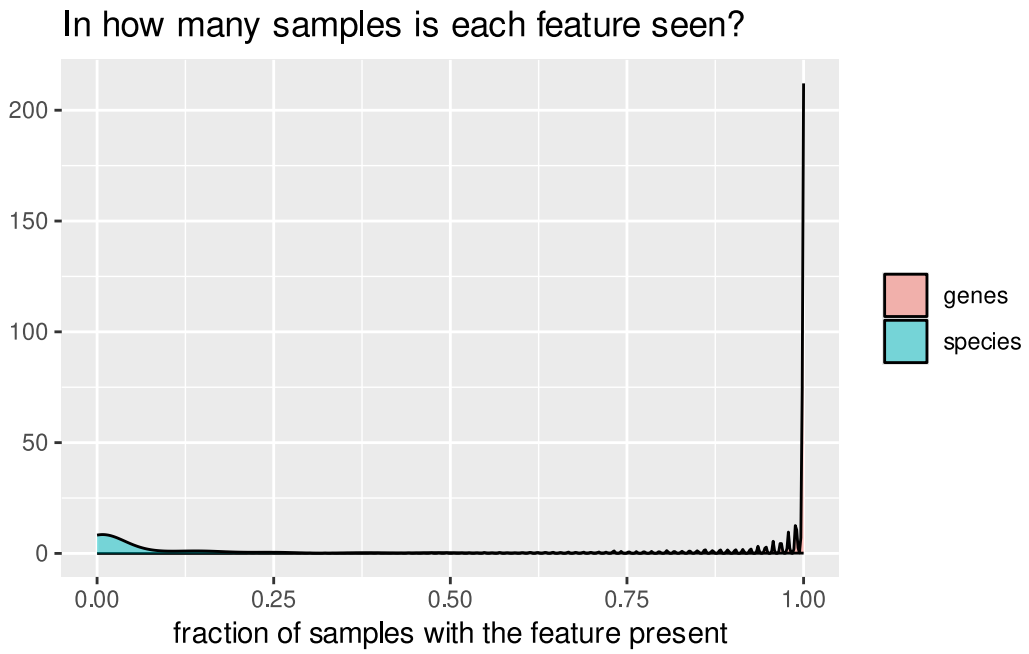



No batch jumps out as different in depth, and both arms appear in both batches. A real check for batch effects in expression profiles comes later (ordination coloured by batch); this only rules out crude depth differences.

## 8. How prevalent are the features?

```
prev_sp <- rowSums(sp > 0) / ncol(sp)
prev_ge <- rowSums(ge > 0) / ncol(ge)

data.frame(prevalence = c(prev_sp, prev_ge),
           table = rep(c("species", "genes"), c(length(prev_sp), length(prev_ge))))
|>
  ggplot(aes(prevalence, fill = table)) +
  geom_density(alpha = 0.5) +
  labs(title = "In how many samples is each feature seen?",
       x = "fraction of samples with the feature present", y = NULL, fill = NULL)
```




Most species are rare (72 % of them appear in under 10 % of samples) — typical for microbial data, and a reminder that feature filtering (`min_samples`) will discard a lot before any differential-abundance test. Genes, in contrast, are dense: nearly all are detected in most samples, because sequencing 14 M reads into 14,226 well-known genes rarely leaves one empty.

## 9. First look: ordination

Ordination gives every sample one point on a map: samples with similar feature profiles sit close together. It is the strongest early check that the data has real structure, and that no sample sits wildly alone. We colour the points by **community**, using the same rule as notebook 02 — a sample is *lactobacillus* when the genus *Lactobacillus* accounts for at least half of its species reads, otherwise *anaerobe*.

```
# Same rule as notebook 02, applied to the group_b species table.
lacto_taxa <- d$taxonomy |> filter(Genus == "Lactobacillus") |> pull(taxon)
qc$lacto_fraction <- colSums(sp[lacto_taxa, , drop = FALSE]) / qc$lib_species
qc$community <- ifelse(qc$lacto_fraction >= 0.5, "lactobacillus", "anaerobe")

knitr::kable(qc |> count(community), caption = "Samples per community label")
```

| community | n |
|---|---|
| anaerobe | 45 |
| lactobacillus | 48 |

```r
library(microViz)
library(phyloseq)

ps_species <- as_counts(dataset = "group_b", table = "microbiome", data = d)
sd <- as(sample_data(ps_species), "data.frame")
sd$community <- qc$community[sd$sample_id]
sample_data(ps_species) <- sample_data(sd)

# One knob each: transform -> distance -> ordination.
ord_bray    <- ps_species |> tax_transform("identity") |> dist_calc("bray")    |
> ord_calc("PCoA")
ord_jaccard <- ps_species |> tax_transform("binary")   |> dist_calc("jaccard")
|> ord_calc("PCoA")
ord_clr     <- ps_species |> tax_transform("clr", zero_replace = "halfmin") |>
dist_calc("euclidean") |> ord_calc("PCoA")

plot_ord <- function(ord, name) {
  eig <- attr(ord, "ord")$CA$eig
  sh  <- round(100 * eig / sum(eig), 1)
  ord_plot(ord, axes = 1:2, color = "community") +
    labs(title = name,
         x = sprintf("Axis 1 (%s%%)", sh[1]),
         y = sprintf("Axis 2 (%s%%)", sh[2]))
}
scree_shares <- function(ord, name) {
  eig <- attr(ord, "ord")$CA$eig[1:8]
  tibble(distance = name, axis = seq_along(eig), share = eig / sum(eig))
}
scree <- bind_rows(
  scree_shares(ord_bray, "Bray-Curtis"),
  scree_shares(ord_jaccard, "Jaccard"),
  scree_shares(ord_clr, "CLR + Euclidean")
)

wrap_plots(
  plot_ord(ord_bray, "Bray-Curtis"),
  plot_ord(ord_jaccard, "Jaccard"),
  plot_ord(ord_clr, "CLR + Euclidean"),
  ncol = 3
) / ggplot(scree, aes(axis, share)) +
  geom_col(fill = "grey40") +
  facet_wrap(~ distance) +
  labs(title = "Scree: share of variation held by the first 8 axes",
```

```
      x = "Axis", y = "Share of variation") +
  theme_bw()
```

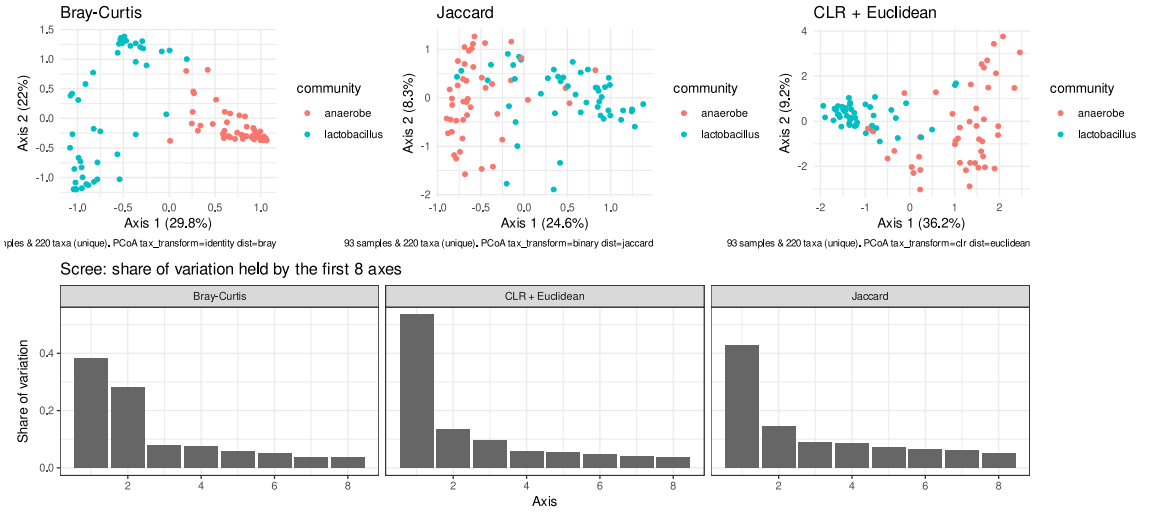



Figure 1: Species-table PCoA three ways (Bray-Curtis, Jaccard, CLR + Euclidean), coloured by community label, with the share of variation held by the first 8 axes underneath.

The split is 48 *lactobacillus* and 45 *anaerobe* samples. The two communities separate along axis 1 in all three panels — most cleanly with **CLR + Euclidean**, where the first axis alone holds 36 % of the variation. That the picture survives three quite different distances (abundance-based, presence/absence-based, and log-ratio-based) is reassuring: the structure is in the data, not an artefact of one distance choice. No sample sits in obvious isolation, so none of the flagged low-depth samples needs dropping at this stage.

**Sanity check.** For any sample, add up the rows of `02_microbiome_counts.csv` whose genus (per `03_microbiome_taxonomy.csv`) is *Lactobacillus*, divide by the column sum, and compare with `qc$lacto_fraction` for that sample — e.g. try `B001`.

# Verdict

- **Structure:** all four tables (metadata, species, genes, cytokines) agree on the same 93 samples and 93 participants, in the same order; counts are non-negative integers. No structural problems.
- **Balance:** 28 pill / 39 injectable / 26 ring, both sequencing batches present in every arm. Baseline variables are balanced across arms except a small difference in vaginal pH ($p = 0.02$) worth remembering at analysis time.
- **Missing data:** the metadata is 100 % complete. `B018` and `B029` lack all cytokines (keep for the rest; drop them for cytokine analyses). No other missing values anywhere.
- **Flagged samples:** nine low-depth microbiome samples (still usable); `B030` is a near-monoculture of *L. iners* (real biology, watch it in ordinations); `B093` carries heavy blood signal in its transcriptome (remember when interpreting it).

- **Nothing so broken that it must be removed before analysis.**

## Choices made

- Flagged low-depth samples at **below half the median library size**, rather than a fixed read count, so the rule adapts to the table's own spread; a fixed floor (e.g. 10,000 reads) would be more standard in published microbiome pipelines.
- Did **not remove any samples** in the QC document itself — removal decisions are deferred to the analysis documents, where we can see whether a flagged sample actually drives a result.
- Did **not impute the 30 missing cytokine values** — complete-case analysis (dropping the two affected samples only for cytokine questions) is simpler and safer.
- Used **prevalence curves** to preview feature filtering but applied no filter here; the filter thresholds belong to each analysis's "Choices made".
- Checked batch only via **library size** — the first-look ordination above is coloured by community, not batch; ordinations coloured by batch, and on the gene table, still belong to the analysis documents.
- Computed the ordination **inside this document** rather than embedding the pre-made `ordination_three_ways.png`, which turned out to have been built from the instructional cohort (471 samples) instead of Study B. Rebuilding here means the figure, its numbers and this document all come from the same 93 samples.
- Defined community **inside this document** with the notebook 02 rule (Lactobacillus ≥ 50 % of species reads) rather than reading `data/my_sample_groups.csv`, which only holds labels for the instructional cohort; a stricter rule (e.g. 60 %, or a third group) is the alternative.
- Ran the ordination on the **unfiltered species table**; filtering rare features first (as the analysis documents will) is the alternative and may sharpen the picture.
- For the baseline check we used **Kruskal–Wallis** for Nugent score and pH (non-normal, three groups) and **Fisher's exact test** for BV and STI (small counts in some cells); a chi-squared test would be the alternative, but some expected counts are too small for it to be trustworthy. We report the pH difference rather than adjust for it here — the decision to include pH as a covariate belongs to each analysis document's "Choices made".